Data Hygiene Audit Report

sample_messy_data.xlsx — 2026-07-27 16:58:34

Health Score: 42/100 — Significant Issues

Total Issues	High	Medium	Low
59	23	20	16

Sheet: Customers

30 rows × 10 columns

CustomerID (id) — Missing: 0/30 (0.0%)

30 distinct | 100.0% unique | avg len 7.9

[High] Mixed ID formats — e.g. "29 coded (e.g. CUST-001) vs 1 bare numbers"

Why this matters: Inconsistent ID formats break joins, lookups, and deduplication — a key stored as "CUST-001" won't match "1027", so related records silently fail to link.

Suggested fix: Identify rows with inconsistent ID formats in "CustomerID" (29 coded (e.g. CUST-001) vs 1 bare numbers)

```
df["_CustomerID_format"] = df["CustomerID"].apply( lambda x: "coded" if isinstance(x, str) and "-" in x else "numeric" )
```

FirstName (name) — Missing: 4/30 (13.3%)

21 distinct | 80.8% unique | avg len 4.7

[Low] High missing rate — 4 of 30 values missing (13.3%)

Why this matters: High rates of missing data reduce the reliability of any analysis built on this field. Missing values can skew averages, break joins between tables, and cause downstream systems to error out or produce incomplete results.

Suggested fix: Fill 4 missing values in "FirstName" (13.3%)

```
df["FirstName"] = df["FirstName"].fillna(df["FirstName"].mode()[0])
```

[High] Code/ID stuffed in name field — e.g. "REF-4421"

Why this matters: When a field is used for something other than its intended purpose — like storing reference codes in a name field or text in a currency field — it corrupts both the misused field and whatever field should have held that data. This makes the data unreliable for any analysis.

Suggested fix: Flag misused values in "FirstName": Code/ID stuffed in name field

```
# Review rows where FirstName contains unexpected data mask = df["FirstName"].notna() suspect = df.loc[mask, "FirstName"]
```

[Medium] Placeholder — "Test" appears 3 times (11.5%)

Why this matters: Placeholder values ("Test", "N/A", "TBD") that persist in production data inflate counts, skew averages, and create phantom records. They often indicate incomplete data entry or inadequate validation at the point of capture.

Suggested fix: Replace 3 placeholder values ("Test") in "FirstName" with NaN for proper missing-data handling

```
import numpy as np df["FirstName"] = df["FirstName"].replace("Test", np.nan)
```

[Low] Placeholder — "TBD" appears 1 times (3.8%)

Why this matters: Placeholder values ("Test", "N/A", "TBD") that persist in production data inflate counts, skew averages, and create phantom records. They often indicate incomplete data entry or inadequate validation at the point of capture.

Suggested fix: Replace 1 placeholder values ("TBD") in "FirstName" with NaN for proper missing-data handling

```
import numpy as np df["FirstName"] = df["FirstName"].replace("TBD", np.nan)
```

LastName (name) — Missing: 4/30 (13.3%)

20 distinct | 76.9% unique | avg len 5.7

[Low] High missing rate — 4 of 30 values missing (13.3%)

Why this matters: High rates of missing data reduce the reliability of any analysis built on this field. Missing values can skew averages, break joins between tables, and cause downstream systems to error out or produce incomplete results.

Suggested fix: Fill 4 missing values in "LastName" (13.3%)

```
df["LastName"] = df["LastName"].fillna(df["LastName"].mode()[0])
```

[Low] Placeholder — "TBD" appears 1 times (3.8%)

Why this matters: Placeholder values ("Test", "N/A", "TBD") that persist in production data inflate counts, skew averages, and create phantom records. They often indicate incomplete data entry or inadequate validation at the point of capture.

Suggested fix: Replace 1 placeholder values ("TBD") in "LastName" with NaN for proper missing-data handling

```
import numpy as np df["LastName"] = df["LastName"].replace("TBD", np.nan)
```

[Medium] Suspicious repetition — "User" appears 3 times (11.5%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 3 rows where "LastName" = "User" (11.5%) for manual review

```
df["_LastName_review"] = ( df["LastName"] == "User" )
```

[Medium] Suspicious repetition — "Doe" appears 3 times (11.5%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 3 rows where "LastName" = "Doe" (11.5%) for manual review

```
df["_LastName_review"] = ( df["LastName"] == "Doe" )
```

Email (email) — Missing: 4/30 (13.3%)

21 distinct | 80.8% unique | avg len 16.5

[Low] High missing rate — 4 of 30 values missing (13.3%)

Why this matters: High rates of missing data reduce the reliability of any analysis built on this field. Missing values can skew averages, break joins between tables, and cause downstream systems to error out or produce incomplete results.

Suggested fix: Fill 4 missing values in "Email" (13.3%)

```
df["Email"] = df["Email"].fillna(df["Email"].mode()[0])
```

[High] Invalid email format — e.g. "henrylee"

Why this matters: When a field is used for something other than its intended purpose — like storing reference codes in a name field or text in a currency field — it corrupts both the misused field and whatever field should have held that data. This makes the data unreliable for any analysis.

Suggested fix: Flag invalid email addresses in "Email" for correction

```
email_re = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$" df["_Email_invalid"] =
~df["Email"].str.match( email_re, na=False )
```

[High] Invalid email format — e.g. "nancy w@example.com"

Why this matters: When a field is used for something other than its intended purpose — like storing reference codes in a name field or text in a currency field — it corrupts both the misused field and whatever field should have held that data. This makes the data unreliable for any analysis.

Suggested fix: Flag invalid email addresses in "Email" for correction

```
email_re = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$" df["_Email_invalid"] =
~df["Email"].str.match( email_re, na=False )
```

[Medium] Suspicious repetition — "test@test.com" appears 3 times (11.5%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 3 rows where "Email" = "test@test.com" (11.5%) for manual review

```
df["_Email_review"] = ( df["Email"] == "test@test.com" )
```

Phone (phone) — Missing: 4/30 (13.3%)

19 distinct | 73.1% unique | avg len 12.6

[Low] High missing rate — 4 of 30 values missing (13.3%)

Why this matters: High rates of missing data reduce the reliability of any analysis built on this field. Missing values can skew averages, break joins between tables, and cause downstream systems to error out or produce incomplete results.

Suggested fix: Fill 4 missing values in "Phone" (13.3%)

```
df["Phone"] = df["Phone"].fillna(df["Phone"].mode()[0])
```

[High] Mixed phone formats — 17 of 26 values deviate from (XXX) XXX-XXXX

Format	Count
(XXX) XXX-XXXX	9
XXX-XXX-XXXX	8
(non-standard)	3
+1-XXX-XXX-XXXX	2
XXXXXXXXXX	1
XXX.XXX.XXXX	1
XXX XXX XXXX	1
(XXX)XXXXXXX	1

Why this matters: Inconsistent phone formats break deduplication, prevent reliable search/lookup, and cause issues with automated dialers or SMS systems. Two records for the same person may appear as different contacts if one says "(555) 123-4567" and another says "5551234567".

Suggested fix: Standardize all phone numbers in "Phone" to (XXX) XXX-XXXX format

```
digits = df["Phone"].str.replace(r"\D", "", regex=True) digits = digits.str.slice(-10) df["Phone"] = ( "(" + digits.str[:3] + ") " + digits.str[3:6] + "-" + digits.str[6:] )
```

[Low] Placeholder — "TBD" appears 1 times (3.8%)

Why this matters: Placeholder values ("Test", "N/A", "TBD") that persist in production data inflate counts, skew averages, and create phantom records. They often indicate incomplete data entry or inadequate validation at the point of capture.

Suggested fix: Replace 1 placeholder values ("TBD") in "Phone" with NaN for proper missing-data handling

```
import numpy as np df["Phone"] = df["Phone"].replace("TBD", np.nan)
```

[Medium] Suspicious repetition — "(555) 123-4567" appears 3 times (11.5%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 3 rows where "Phone" = "(555) 123-4567" (11.5%) for manual review

```
df["_Phone_review"] = ( df["Phone"] == "(555) 123-4567" )
```

[Medium] Suspicious repetition — "000-000-0000" appears 3 times (11.5%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 3 rows where "Phone" = "000-000-0000" (11.5%) for manual review

```
df["_Phone_review"] = ( df["Phone"] == "000-000-0000" )
```

[Medium] Suspicious repetition — "555-555-5555" appears 3 times (11.5%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 3 rows where "Phone" = "555-555-5555" (11.5%) for manual review

```
df["_Phone_review"] = ( df["Phone"] == "555-555-5555" )
```

JoinDate (date) — Missing: 4/30 (13.3%)

18 distinct | 69.2% unique | avg len 10.0

[Low] High missing rate — 4 of 30 values missing (13.3%)

Why this matters: High rates of missing data reduce the reliability of any analysis built on this field. Missing values can skew averages, break joins between tables, and cause downstream systems to error out or produce incomplete results.

Suggested fix: Fill 4 missing values in "JoinDate" (13.3%)

```
df["JoinDate"] = df["JoinDate"].fillna(df["JoinDate"].mode()[0])
```

[High] Mixed date formats — 12 of 26 values deviate from YYYY-MM-DD

Format	Count
YYYY-MM-DD	14
MM/DD/YYYY	3
Mon DD, YYYY	3
(non-standard)	1
YYYY/MM/DD	1
M/D/YYYY	1
DD-Mon-YYYY	1
YYYY.MM.DD	1
MM-DD-YYYY	1

Why this matters: Mixed date formats cause sorting failures, broken filters, and incorrect calculations. A date stored as text ("Jan 15, 2023") won't sort chronologically next to "2023-01-15". Downstream tools, APIs, and reports will misparse or reject inconsistent dates.

Suggested fix: Standardize all dates in "JoinDate" to YYYY-MM-DD format

```
df["JoinDate"] = pd.to_datetime( df["JoinDate"], format="mixed", dayfirst=False
).dt.strftime("%Y-%m-%d")
```

[Low] Placeholder — "TBD" appears 1 times (3.8%)

Why this matters: Placeholder values ("Test", "N/A", "TBD") that persist in production data inflate counts, skew averages, and create phantom records. They often indicate incomplete data entry or inadequate validation at the point of capture.

Suggested fix: Replace 1 placeholder values ("TBD") in "JoinDate" with NaN for proper missing-data handling

```
import numpy as np df["JoinDate"] = df["JoinDate"].replace("TBD", np.nan)
```

[High] Suspicious repetition — "2023-01-01" appears 6 times (23.1%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 6 rows where "JoinDate" = "2023-01-01" (23.1%) for manual review

```
df["_JoinDate_review"] = ( df["JoinDate"] == "2023-01-01" )
```

[Medium] Suspicious repetition — "2023-01-15" appears 3 times (11.5%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 3 rows where "JoinDate" = "2023-01-15" (11.5%) for manual review

```
df["_JoinDate_review"] = ( df["JoinDate"] == "2023-01-15" )
```

AccountBalance (currency) — Missing: 4/30 (13.3%)

18 distinct | 69.2% unique | avg len 7.5 | range -500.0–11000.0

[Low] High missing rate — 4 of 30 values missing (13.3%)

Why this matters: High rates of missing data reduce the reliability of any analysis built on this field. Missing values can skew averages, break joins between tables, and cause downstream systems to error out or produce incomplete results.

Suggested fix: Fill 4 missing values in "AccountBalance" (13.3%)

```
df["AccountBalance"] = df["AccountBalance"].fillna(df["AccountBalance"].mode()[0])
```

[High] Mixed currency formats — 11 of 26 values deviate from $X,XXX.XX

Format	Count
$X,XXX.XX	15
X,XXX.XX (no symbol)	4
$X,XXX	3
(non-standard)	2
$X,XXX.XX USD	2

Why this matters: Mixed currency formats ("$1,250.00" vs "1250" vs "five thousand") prevent accurate aggregation and comparison. Summing a column with text-formatted currency returns errors or silently drops values, leading to wrong totals in financial reports.

Suggested fix: Standardize all currency values in "AccountBalance" to numeric format

```
df["AccountBalance"] = ( df["AccountBalance"].str.replace(r"[^\d.]", "", regex=True) .astype(float) )
```

[High] Text in currency field — e.g. "TBD"

Why this matters: When a field is used for something other than its intended purpose — like storing reference codes in a name field or text in a currency field — it corrupts both the misused field and whatever field should have held that data. This makes the data unreliable for any analysis.

Suggested fix: Flag non-numeric values in currency field "AccountBalance"

```
df["_AccountBalance_invalid"] = ~df["AccountBalance"].str.match( r"^[\$\d,\.\-]+$", na=False )
```

[High] Text in currency field — e.g. "five thousand"

Why this matters: When a field is used for something other than its intended purpose — like storing reference codes in a name field or text in a currency field — it corrupts both the misused field and whatever field should have held that data. This makes the data unreliable for any analysis.

Suggested fix: Flag non-numeric values in currency field "AccountBalance"

```
df["_AccountBalance_invalid"] = ~df["AccountBalance"].str.match( r"^[\$\d,\.\-]+$", na=False )
```

[Low] Placeholder — "TBD" appears 1 times (3.8%)

Why this matters: Placeholder values ("Test", "N/A", "TBD") that persist in production data inflate counts, skew averages, and create phantom records. They often indicate incomplete data entry or inadequate validation at the point of capture.

Suggested fix: Replace 1 placeholder values ("TBD") in "AccountBalance" with NaN for proper missing-data handling

```
import numpy as np df["AccountBalance"] = df["AccountBalance"].replace("TBD", np.nan)
```

[High] Suspicious repetition — "$0.00" appears 6 times (23.1%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 6 rows where "AccountBalance" = "$0.00" (23.1%) for manual review

```
df["_AccountBalance_review"] = ( df["AccountBalance"] == "$0.00" )
```

[Medium] Suspicious repetition — "$1,250.00" appears 3 times (11.5%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 3 rows where "AccountBalance" = "$1,250.00" (11.5%) for manual review

```
df["_AccountBalance_review"] = ( df["AccountBalance"] == "$1,250.00" )
```

Status (categorical) — Missing: 4/30 (13.3%)

8 distinct | 30.8% unique | avg len 5.7

[Low] High missing rate — 4 of 30 values missing (13.3%)

Why this matters: High rates of missing data reduce the reliability of any analysis built on this field. Missing values can skew averages, break joins between tables, and cause downstream systems to error out or produce incomplete results.

Suggested fix: Fill 4 missing values in "Status" (13.3%)

```
df["Status"] = df["Status"].fillna(df["Status"].mode()[0])
```

[High] Inconsistent casing in categorical values — e.g. "ACTIVE / Active / active"

Why this matters: When a field is used for something other than its intended purpose — like storing reference codes in a name field or text in a currency field — it corrupts both the misused field and whatever field should have held that data. This makes the data unreliable for any analysis.

Suggested fix: Flag misused values in "Status": Inconsistent casing in categorical values

```
# Review rows where Status contains unexpected data mask = df["Status"].notna() suspect = df.loc[mask, "Status"]
```

[Low] Placeholder — "TBD" appears 1 times (3.8%)

Why this matters: Placeholder values ("Test", "N/A", "TBD") that persist in production data inflate counts, skew averages, and create phantom records. They often indicate incomplete data entry or inadequate validation at the point of capture.

Suggested fix: Replace 1 placeholder values ("TBD") in "Status" with NaN for proper missing-data handling

```
import numpy as np df["Status"] = df["Status"].replace("TBD", np.nan)
```

[High] Suspicious repetition — "Active" appears 18 times (69.2%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 18 rows where "Status" = "Active" (69.2%) for manual review

```
df["_Status_review"] = ( df["Status"] == "Active" )
```

ZipCode (zipcode) — Missing: 4/30 (13.3%)

19 distinct | 73.1% unique | avg len 5.1

[Low] High missing rate — 4 of 30 values missing (13.3%)

Why this matters: High rates of missing data reduce the reliability of any analysis built on this field. Missing values can skew averages, break joins between tables, and cause downstream systems to error out or produce incomplete results.

Suggested fix: Fill 4 missing values in "ZipCode" (13.3%)

```
df["ZipCode"] = df["ZipCode"].fillna(df["ZipCode"].mode()[0])
```

[Medium] Placeholder — "00000" appears 3 times (11.5%)

Why this matters: Placeholder values ("Test", "N/A", "TBD") that persist in production data inflate counts, skew averages, and create phantom records. They often indicate incomplete data entry or inadequate validation at the point of capture.

Suggested fix: Replace 3 placeholder values ("00000") in "ZipCode" with NaN for proper missing-data handling

```
import numpy as np df["ZipCode"] = df["ZipCode"].replace("00000", np.nan)
```

[Low] Placeholder — "TBD" appears 1 times (3.8%)

Why this matters: Placeholder values ("Test", "N/A", "TBD") that persist in production data inflate counts, skew averages, and create phantom records. They often indicate incomplete data entry or inadequate validation at the point of capture.

Suggested fix: Replace 1 placeholder values ("TBD") in "ZipCode" with NaN for proper missing-data handling

```
import numpy as np df["ZipCode"] = df["ZipCode"].replace("TBD", np.nan)
```

[Medium] Suspicious repetition — "30301" appears 3 times (11.5%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 3 rows where "ZipCode" = "30301" (11.5%) for manual review

```
df["_ZipCode_review"] = ( df["ZipCode"] == "30301" )
```

[Medium] Suspicious repetition — "12345" appears 3 times (11.5%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 3 rows where "ZipCode" = "12345" (11.5%) for manual review

```
df["_ZipCode_review"] = ( df["ZipCode"] == "12345" )
```

Notes (freetext) — Missing: 15/30 (50.0%)

11 distinct | 73.3% unique | avg len 12.3

[Medium] High missing rate — 15 of 30 values missing (50.0%)

Why this matters: High rates of missing data reduce the reliability of any analysis built on this field. Missing values can skew averages, break joins between tables, and cause downstream systems to error out or produce incomplete results.

Suggested fix: Fill 15 missing values in "Notes" (50.0%)

```
# Option A: Fill with most common value df["Notes"] = df["Notes"].fillna(df["Notes"].mode()[0]) #
Option B: Fill with a sentinel df["Notes"] = df["Notes"].fillna("MISSING")
```

[Medium] Placeholder — "TEST" appears 3 times (20.0%)

Why this matters: Placeholder values ("Test", "N/A", "TBD") that persist in production data inflate counts, skew averages, and create phantom records. They often indicate incomplete data entry or inadequate validation at the point of capture.

Suggested fix: Replace 3 placeholder values ("TEST") in "Notes" with NaN for proper missing-data handling

```
import numpy as np df["Notes"] = df["Notes"].replace("TEST", np.nan)
```

[Medium] Placeholder — "TBD" appears 1 times (6.7%)

Why this matters: Placeholder values ("Test", "N/A", "TBD") that persist in production data inflate counts, skew averages, and create phantom records. They often indicate incomplete data entry or inadequate validation at the point of capture.

Suggested fix: Replace 1 placeholder values ("TBD") in "Notes" with NaN for proper missing-data handling

```
import numpy as np df["Notes"] = df["Notes"].replace("TBD", np.nan)
```

[Medium] Suspicious repetition — "Preferred customer" appears 3 times (20.0%)

Why this matters: When the same value appears far more often than expected, it may indicate a default value that was never updated, a copy-paste error, or a system glitch that stamped the same data across multiple records.

Suggested fix: Flag 3 rows where "Notes" = "Preferred customer" (20.0%) for manual review

```
df["_Notes_review"] = ( df["Notes"] == "Preferred customer" )
```

Phantom & Exact Duplicates

[Medium] Phantom Duplicate — 3 rows: 2, 8, 9

Why this matters: These records look different on the surface (different casing, extra spaces, punctuation variations) but represent the same entity. They cause inflated counts, split transaction histories, and duplicate outreach — problems that compound over time.

Suggested fix: Normalize and deduplicate 3 phantom duplicate rows (rows 2, 8, 9)

[Medium] Phantom Duplicate — 2 rows: 3, 10

Why this matters: These records look different on the surface (different casing, extra spaces, punctuation variations) but represent the same entity. They cause inflated counts, split transaction histories, and duplicate outreach — problems that compound over time.

Suggested fix: Normalize and deduplicate 2 phantom duplicate rows (rows 3, 10)

[High] Exact Duplicate — 3 rows: 11, 12, 13

Why this matters: Exact duplicate rows are the clearest sign of a data quality issue — they can result from double-submissions, ETL failures, or missing unique constraints. Every duplicate inflates counts and distorts any metric built on this data.

Suggested fix: Remove 3 exact duplicate rows (rows 11, 12, 13)

[High] Phantom Duplicate — 4 rows: 14, 19, 20, 21

Why this matters: These records look different on the surface (different casing, extra spaces, punctuation variations) but represent the same entity. They cause inflated counts, split transaction histories, and duplicate outreach — problems that compound over time.

Suggested fix: Normalize and deduplicate 4 phantom duplicate rows (rows 14, 19, 20, 21)

[High] Exact Duplicate — 2 rows: 25, 27

Why this matters: Exact duplicate rows are the clearest sign of a data quality issue — they can result from double-submissions, ETL failures, or missing unique constraints. Every duplicate inflates counts and distorts any metric built on this data.

Suggested fix: Remove 2 exact duplicate rows (rows 25, 27)

Sheet: Orders

10 rows × 6 columns

OrderDate (date) — Missing: 0/10 (0.0%)

8 distinct | 80.0% unique | avg len 10.0

[High] Mixed date formats — 4 of 10 values deviate from YYYY-MM-DD

Format	Count
YYYY-MM-DD	6
MM/DD/YYYY	1
Mon DD, YYYY	1
M/D/YYYY	1
YYYY/MM/DD	1

Why this matters: Mixed date formats cause sorting failures, broken filters, and incorrect calculations. A date stored as text ("Jan 15, 2023") won't sort chronologically next to "2023-01-15". Downstream tools, APIs, and reports will misparse or reject inconsistent dates.

Suggested fix: Standardize all dates in "OrderDate" to YYYY-MM-DD format

```
df["OrderDate"] = pd.to_datetime( df["OrderDate"], format="mixed", dayfirst=False ).dt.strftime("%Y-%m-%d")
```

Amount (currency) — Missing: 0/10 (0.0%)

8 distinct | 80.0% unique | avg len 5.5 | range 0.0–450.0

[Medium] Mixed currency formats — 3 of 10 values deviate from $X,XXX.XX

Format	Count
$X,XXX.XX	7
X,XXX (bare number)	2
$X,XXX	1

Why this matters: Mixed currency formats ("$1,250.00" vs "1250" vs "five thousand") prevent accurate aggregation and comparison. Summing a column with text-formatted currency returns errors or silently drops values, leading to wrong totals in financial reports.

Suggested fix: Standardize all currency values in "Amount" to numeric format

```
df["Amount"] = ( df["Amount"].str.replace(r"[^\d.]", "", regex=True) .astype(float) )
```

ShipDate (date) — Missing: 2/10 (20.0%)

6 distinct | 75.0% unique | avg len 9.9

[Low] High missing rate — 2 of 10 values missing (20.0%)

Why this matters: High rates of missing data reduce the reliability of any analysis built on this field. Missing values can skew averages, break joins between tables, and cause downstream systems to error out or produce incomplete results.

Suggested fix: Fill 2 missing values in "ShipDate" (20.0%)

```
df["ShipDate"] = df["ShipDate"].fillna(df["ShipDate"].mode()[0])
```

[High] Mixed date formats — 3 of 8 values deviate from YYYY-MM-DD

Format	Count
YYYY-MM-DD	5
MM/DD/YYYY	1
Mon DD, YYYY	1

M/D/YYYY	1

Why this matters: Mixed date formats cause sorting failures, broken filters, and incorrect calculations. A date stored as text ("Jan 15, 2023") won't sort chronologically next to "2023-01-15". Downstream tools, APIs, and reports will misparse or reject inconsistent dates.

Suggested fix: Standardize all dates in "ShipDate" to YYYY-MM-DD format

```
df["ShipDate"] = pd.to_datetime( df["ShipDate"], format="mixed", dayfirst=False
).dt.strftime("%Y-%m-%d")
```

Status (categorical) — Missing: 0/10 (0.0%)

7 distinct | 70.0% unique | avg len 7.0

[High] Inconsistent casing in categorical values — e.g. "Shipped / shipped"

Why this matters: When a field is used for something other than its intended purpose — like storing reference codes in a name field or text in a currency field — it corrupts both the misused field and whatever field should have held that data. This makes the data unreliable for any analysis.

Suggested fix: Flag misused values in "Status": Inconsistent casing in categorical values

```
# Review rows where Status contains unexpected data mask = df["Status"].notna() suspect =
df.loc[mask, "Status"]
```

[High] Inconsistent casing in categorical values — e.g. "Delivered / delivered"

Why this matters: When a field is used for something other than its intended purpose — like storing reference codes in a name field or text in a currency field — it corrupts both the misused field and whatever field should have held that data. This makes the data unreliable for any analysis.

Suggested fix: Flag misused values in "Status": Inconsistent casing in categorical values

```
# Review rows where Status contains unexpected data mask = df["Status"].notna() suspect =
df.loc[mask, "Status"]
```

[High] Inconsistent casing in categorical values — e.g. "PENDING / Pending"

Why this matters: When a field is used for something other than its intended purpose — like storing reference codes in a name field or text in a currency field — it corrupts both the misused field and whatever field should have held that data. This makes the data unreliable for any analysis.

Suggested fix: Flag misused values in "Status": Inconsistent casing in categorical values

```
# Review rows where Status contains unexpected data mask = df["Status"].notna() suspect =
df.loc[mask, "Status"]
```

[Medium] Placeholder — "Test" appears 2 times (20.0%)

Why this matters: Placeholder values ("Test", "N/A", "TBD") that persist in production data inflate counts, skew averages, and create phantom records. They often indicate incomplete data entry or inadequate validation at the point of capture.

Suggested fix: Replace 2 placeholder values ("Test") in "Status" with NaN for proper missing-data handling

```
import numpy as np df["Status"] = df["Status"].replace("Test", np.nan)
```

Phantom & Exact Duplicates

[High] Exact Duplicate — 2 rows: 5, 6

Why this matters: Exact duplicate rows are the clearest sign of a data quality issue — they can result from double-submissions, ETL failures, or missing unique constraints. Every duplicate inflates counts and distorts any metric built on this data.

Suggested fix: Remove 2 exact duplicate rows (rows 5, 6)

[High] Exact Duplicate — 2 rows: 7, 8

Why this matters: Exact duplicate rows are the clearest sign of a data quality issue — they can result from double-submissions, ETL failures, or missing unique constraints. Every duplicate inflates counts and distorts any metric built on this data.

Suggested fix: Remove 2 exact duplicate rows (rows 7, 8)